CRIMINAL RECORD AND SEARCH SYSTEM USING STRING MATCHING ALGORITHM

MODULE: DNA / FINGERPRINT PATTERN SEARCH

A CAPSTONE PROJECT REPORT

Submitted in the partial fulfilment for the Course of

DATA STRUCTURES & ALGORITHMS

to the award of the degree of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE ENGINEERING

Submitted by

A Poojitha (192511230)

Sabitha V

M Nandhini (192472318)

Under the Supervision of

Dr. T Rajesh Kumar

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences Chennai-602105

SEPTEMBER 2026

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled "Criminal Record and Search System Using String Matching Algorithm", with the module component "DNA / FINGERPRINT PATTERN SEARCH", has been carried out by A Poojitha (192511230), Sabitha V, and M Nandhini (192472318) under the supervision of Dr. T Rajesh Kumar and is submitted in partial fulfilment of the requirements for the Capstone Project course of the B.Tech / B.E. Computer Science Engineering programme at simats engineering collage.

COURSE COORDINATOR

Dr.C.Anusuya

Department of computer and

engineering SIMATS

Engineering, Chennai – 602105.

COURSE FACULTY / GUIDE

Dr. Rajesh kumar

Department of computer and

engineering SIMATS

Engineering, Chennai – 602105.

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

DECLARATION

We, A Poojitha, Sabitha V, and M Nandhini, of the Department of Computer Science and Engineering, [Institution Name], hereby declare that the Capstone Project Work entitled "Criminal Record and Search System Using String Matching Algorithm" is the result of our own bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, datasets, and other materials used in the project have been appropriately acknowledged. We further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated. Any external tools, software, datasets, computational resources, or AI-assisted tools used during the project have been appropriately disclosed. We confirm that all applicable ethical, academic, institutional, and professional requirements have been duly followed throughout the planning, execution, analysis, and documentation of the project.

Place:Chennai

Date: 03-09-2026

Signature of the Students with Name

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

Student Name / Reg. No.	Specific Responsibilities	Design & Development	Testing & Analysis	Report Contribution	% Contribution
A Poojitha (192511230)	Requirement analysis, database schema for criminal records, alias data model	Designed the record/alias schema; implemented phonetic matching (Soundex, Metaphone)	Verified phonetic-match accuracy against a labelled alias test set	Sections 3.1–3.3, 4.1–4.3	34%
Sabitha V	String-matching algorithm implementation (KMP, Boyer–Moore), search indexing and query interface	Implemented exact/substring search engine and query API; integrated with record database	Performed exact-match testing and query-latency measurement	Sections 3.4–3.5, 4.4–4.6	33%
M Nandhini (192472318)	Edit-distance scoring, ranking/threshold logic, evaluation	Implemented Levenshtein distance scoring and ranked threshold filtering	Performed precision/recall and false positive/false negative analysis; threshold tuning	Sections 3.6–3.7, 4.7–4.8, Chapter 5	33%

ABSTRACT

Criminal record management and retrieval remains a critical function of law-enforcement and judicial information systems, yet conventional record-search tools that rely on exact string equality routinely fail when a suspect or offender is recorded under a misspelled name, a transliterated name, or a deliberately used alias. This gap allows records belonging to the same individual to remain fragmented across a database, undermining background checks, case linkage, and repeat-offender identification. This Capstone Project addresses the engineering problem by designing and developing a Criminal Record and Search System that retrieves and ranks matching records using string-matching algorithms, allowing investigators to locate a person's record even when the queried name differs from the stored name due to spelling variation, phonetic similarity, or known alias. The work reported here corresponds to the Alias & Name Matching Module of the project, which forms the core retrieval engine on which the record-management front end and case-linkage reporting depend.

The module combines exact-match indexing for fast lookup with approximate string-matching techniques — edit-distance (Levenshtein) scoring, phonetic encoding (Soundex and Metaphone), and substring search using the Knuth–Morris– Pratt and Boyer–Moore algorithms — to compute a similarity-ranked list of candidate records for a given query name, including any aliases stored against each record.

The methodology combined a review of existing record-search and identity-resolution approaches, requirement analysis with law-enforcement-style use cases, comparative evaluation of candidate string-matching algorithms, and iterative testing of the matching pipeline against a labelled dataset of names and known-alias pairs. Major results include a working alias-matching engine that returns ranked candidate matches within an acceptable similarity threshold, a documented comparison of algorithmic accuracy and performance trade-offs, and a measured improvement in successful record retrieval when alias and fuzzy matching are enabled compared with exact-match only search. The principal conclusion is that combining phonetic encoding with edit-distance scoring, layered on top of an efficient exact/substring search index, provides a practical and extensible approach to alias-aware criminal-record retrieval, and that careful threshold tuning is essential to balance recall (finding true matches, including aliases) against precision (avoiding excessive false positives) in an investigative setting.

TABLE OF CONTENTS

Sl. No.	Title	Page No.
i	Bonafide Certificate	ii
ii	Declaration by the Candidate	iii
iii	Individual Contribution Statement	iv
iv	Abstract	v
v	List of Figures	vi
vi	List of Tables	vii
vii	List of Abbreviations	viii
1	Chapter 1: Introduction and Engineering Problem	1
2	Chapter 2: Literature Review and Existing Solutions	5
3	Chapter 3: Engineering Design & Trade-offs, Sustainability, Ethics, Safety, Risk & Security	9
4	Chapter 4: Implementation, Testing & Results, Project Management	16
5	Chapter 5: Conclusion, Future Work and Reflection	22
	References	24
	Appendices	25

LIST OF FIGURES

Figure No.	Figure Name	Page No.
1.1	Fragmentation of criminal records due to alias / spelling variation (illustrative)	3

2.1	Taxonomy of existing name-matching and record-search approaches	6
3.1	Alias & Name Matching Module architecture — query to ranked results	12
3.2	Matching pipeline flowchart (normalize → phonetic encode → score → rank)	13
4.1	Precision–recall trade-off across similarity thresholds	18
4.2	Algorithm-wise matching-accuracy comparison (top attributes)	20

LIST OF TABLES

Table No.	Table Name	Page No.
2.1	Comparative analysis of existing name/record-matching approaches	7
3.1	Stakeholder / user requirements	9
3.2	Functional and non-functional requirements	10
3.3	Constraints and applicable standards	11
3.4	Alternative algorithm evaluation matrix	13
3.5	Risk register	15
4.1	Tools and technologies used	17
4.2	Test plan and verification	19
4.3	Achievement of objectives	21

LIST OF ABBREVIATIONS / SYMBOLS

Symbol	Description	Unit
DB	Database	—

KMP	Knuth–Morris–Pratt (string-matching algorithm)	—
LD	Levenshtein Distance (edit distance)	—
FN / FP	False Negative / False Positive	—
API	Application Programming Interface	—
SQL	Structured Query Language	—
UI	User Interface	—
ID	Identifier	—
ROC-AUC	Receiver Operating Characteristic – Area Under Curve	—
CSV	Comma-Separated Values	—

Chapter-1

INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

Criminal investigations require accurate and fast identification of suspects using different types of information. Biometric information such as DNA sequences and fingerprint patterns can be useful for identifying individuals and connecting them with existing criminal records. However, when a database contains a large number of records, searching for a particular biometric pattern manually becomes difficult and time-consuming.

The Criminal Record and Search System is developed to improve the process of searching and retrieving criminal information. Module 2 of this project focuses on DNA and fingerprint pattern searching. In this module, an input biometric pattern is compared with patterns stored in the criminal record database to identify possible matches. The main objective is to perform the search quickly and efficiently, especially when dealing with a large number of records.

The Boyer–Moore Algorithm is used in this module as the main pattern-matching technique. It searches for a pattern efficiently by reducing unnecessary comparisons when a mismatch occurs. This makes the algorithm suitable for searching DNA sequences and simplified fingerprint patterns stored in the database. The module aims to provide faster biometric verification and help in identifying possible repeat offenders.

1.2 Need for the Project

In criminal record systems, biometric data can contain long sequences and complex patterns. Comparing these patterns manually requires more time and effort. When the number of stored records increases, the searching process becomes slower and more difficult. Therefore, an efficient searching mechanism is required to compare an input DNA sequence or fingerprint pattern with the patterns stored in the database. The proposed module helps reduce the searching time by applying an efficient string-matching algorithm. Instead of checking every character unnecessarily, the Boyer–Moore Algorithm uses an optimized searching approach to find the required pattern. This can improve the speed of biometric pattern searching and make the verification process more efficient.

1.3 Problem Statement

Large criminal record databases may contain numerous DNA sequences and fingerprint patterns. Searching for a matching biometric pattern using manual methods or simple character-by-character comparison can be time-consuming and inefficient. This may delay the process of identifying suspects

and verifying criminal records.

Therefore, there is a need for an efficient system that can compare an input biometric pattern with stored records and quickly identify possible matches. The proposed DNA/Fingerprint Pattern Search Module addresses this problem by using the Boyer–Moore Algorithm to perform fast pattern matching and improve the efficiency of biometric record searching.

1.4 Project Aim

The aim of this module is to design and implement a DNA/Fingerprint Pattern Search system that compares biometric patterns with stored criminal records and identifies matching patterns using the Boyer–Moore String Matching Algorithm.

1.5 Project Objectives

The main objectives of this module are to store and manage biometric pattern information, accept an input DNA sequence or fingerprint pattern for searching, compare the input pattern with stored criminal records, and identify possible matching records. The module also aims to implement the Boyer–Moore Algorithm for efficient pattern matching, reduce unnecessary comparisons during the search process, and improve the speed of biometric verification. By providing faster pattern searching, the system can assist in identifying repeat offenders and retrieving relevant criminal records more efficiently.

1.6 Scope

The scope of this module includes DNA sequence pattern searching, fingerprint pattern comparison, storage of simplified biometric patterns, and searching for matching patterns in criminal records. The Boyer–Moore Algorithm is used as the primary technique for pattern matching. The module displays the records that contain the searched biometric pattern and supports faster retrieval of possible matching records.

The module focuses only on the pattern-searching process. It does not perform actual DNA laboratory testing or advanced fingerprint image recognition. Instead, biometric information is represented in a suitable sequence or pattern format so that string-matching algorithms can be applied.

1.7 Expected Outcome

The expected outcome of this module is an efficient biometric pattern-searching system that can accept an input DNA sequence or fingerprint pattern and compare it with stored criminal records. The system should identify matching patterns quickly and display the corresponding criminal records.

The module is expected to reduce searching time, improve the efficiency of biometric verification, support large-scale pattern searching, and assist in identifying possible repeat offenders.

CHAPTER 2

LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

Relevant literature and existing solutions were identified through a structured review of established string-matching and approximate-string-matching algorithms, together with an inspection of record-linkage and identity-resolution techniques used in public-records and law-enforcement information systems. Sources were screened for relevance to name-matching and alias resolution specifically, rather than the broader field of general full-text search.

2.2 Review of Existing Work

Exact-match database queries (SQL equality or simple wildcard "LIKE" patterns) remain the most common approach in legacy record-management systems. They are computationally cheap and simple to implement, but are strictly literal: any deviation in spelling, capitalization normalisation aside, or the use of an unrecorded alias causes the query to miss a relevant record entirely.

Exact substring-search algorithms such as Knuth–Morris–Pratt (KMP) and Boyer–Moore improve on naive substring scanning by avoiding redundant character comparisons, achieving linear-time matching in the length of the text and pattern. These algorithms are well suited to fast lookup once a candidate name is known, but on their own they still require the query substring to appear verbatim within the target string, so they do not address phonetic or spelling variation.

Phonetic-encoding algorithms, most notably Soundex and its refinement Metaphone (and Double Metaphone), map a name to a code representing its approximate pronunciation, so that names that sound alike (e.g., "Catherine" and "Kathryn") map to the same or similar codes even when spelled differently. These techniques are inexpensive and effective for common phonetic variation but can produce false positives among unrelated names that happen to share a phonetic code, and their accuracy varies across name origins and languages.

Edit-distance techniques, most notably the Levenshtein distance and related measures such as

Damerau–Levenshtein (which also accounts for transpositions) and Jaro–Winkler similarity (which weights matching prefixes more heavily, a property useful for names), quantify how many single-character insertions, deletions, substitutions (and, for Damerau–Levenshtein, transpositions) are needed to transform one string into another. These measures directly capture typographical errors and minor spelling variants but are more computationally expensive than exact matching and require a similarity threshold to be chosen carefully to balance recall against false positives.

Record-linkage and identity-resolution research in public-records and law-enforcement contexts generally combines several of the above techniques — for example, blocking candidate records with a cheap phonetic key before applying a more expensive edit-distance comparison only within that reduced candidate set — to achieve both scalability and matching accuracy. This layered, multi-algorithm strategy is the paradigm adopted by the present module.

2.3 Comparative Analysis

Existing Approach	**Advantages**	**Limitations**	**Relevance to Proposed Work**
Exact-match SQL query	Very low computational cost;simple to implement	Misses any spelling variation or unrecorded alias entirely	Retained as the fast first pass lookup, not the sole mechanism

Existing Approach	**Advantages**	**Limitations**	**Relevance to Proposed Work**
Substring search (naive / KMP / Boyer–Moore)	Linear-time, efficient exact substring matching	Still requires literal substring presence; no tolerance for spelling variation	KMP and Boyer–Moore adopted for fast candidate name substring search

Phonetic encoding (Soundex / Metaphone)	Cheap; captures 'sounds alike' variation well	Can over-match unrelated names sharing a phonetic code; language-dependent accuracy	Adopted as a blocking/candidate generation step to catch phonetic aliases
Edit distance (Levenshtein /Jaro–Winkler)	Directly models typographical/spelling variation; tunable threshold	Higher computational cost per comparison; threshold sensitivity	Adopted as the final similarity-scoring and ranking step

2.4 Research / Engineering Gap

The review shows that no single string-matching technique adequately addresses both spelling variation and deliberate aliasing while remaining fast enough for interactive search at database scale: exact and substring methods are fast but literal; phonetic methods tolerate pronunciation variation but risk false positives; edit-distance methods are accurate for spelling variation but computationally heavier if applied to every record without a cheaper pre-filter. This leaves an unresolved engineering gap: a layered matching pipeline, purpose-built for alias-aware criminal record retrieval, that combines these techniques with a documented, tunable similarity threshold and a measured precision/recall trade-off.

2.5 Proposed Contribution

This module contributes a layered alias-and-name-matching pipeline that combines KMP/Boyer–Moore substring search for fast exact and near-exact lookup, Soundex/Metaphone phonetic blocking for pronunciation-tolerant candidate generation, and Levenshtein-distance similarity scoring for final ranking together with a documented evaluation of matching accuracy and latency on a labelled name/alias test dataset, directly addressing the accuracyperformance trade-off .

CHAPTER 3

ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

Stakeholder / User Requirements

Stakeholder	**Requirement**
Investigating officer (end user)	Must retrieve a person's record even when the queried name differs slightly from the stored spelling or a known alias is used
Record-management front end (downstream module)	Matching results must be returned as a ranked, scored list with a stable API contract
Project supervisor / assessor	Matching pipeline and evaluation must be reproducible, documented, and ethically scoped
System (latency / performance)	Query response must fit within an interactive search budget suitable for live investigative use

Functional & Non-Functional Requirements

Requirement	**Type**	**Measurable Target**
Store multiple known aliases per record	Functional	≥ 3 alias slots per individual record
Exact/substring name search	Functional	100% recall for exact-spelling queries
Phonetic (sounds-alike) matching	Functional	Correctly matches ≥ 90% of labelled phonetic variant pairs
Edit-distance fuzzy matching	Functional	Correctly matches ≥ 85% of labelled 1–2 character typo pairs
Ranked, threshold-filtered results	Functional	Results ordered by similarity score; configurable cut-off threshold

Search latency	Non-functional	≤ 1 second per query on a 10,000–50,000 record test database
False-positive rate at chosen threshold	Non-functional	≤ 10% of returned candidates unrelated to the true match

3.2 Constraints & Applicable Standards

Standard / Code	Requirement	How Applied in This Project
Institutional data-ethics / academic-integrity policy	Only synthetic or properly de identified/consented data may be used	Test dataset built from synthetic names and publicly documented alias examples; no real criminal records used
Data-protection principles (general, e.g., minimisation, purpose limitation)	Personal data handled must be limited and access-controlled	Test dataset limited to name/alias fields only; no other identifying attributes processed

3.3 Design Specifications

Parameter	Target / Requirement	Unit	Priority
Alias slots per record	≥ 3	aliases/record	High
Matching stages implemented	3 (exact/substring, phonetic, edit-distance)	stages	High
Query latency (test database scale)	≤ 1	s/query	High
Phonetic-match accuracy	≥ 90	%	Medium
Edit-distance-match accuracy	≥ 85	%	Medium
False-positive rate at chosen threshold	≤ 10	%	Medium

3.4 Alternative Solutions & Evaluation

Alternative 1 — Exact-match only: Rely solely on SQL equality/wildcard search against the name and alias fields, with no fuzzy or phonetic tolerance.

Alternative 2 — Single-algorithm fuzzy matching: Apply only an edit-distance measure (e.g., Levenshtein) across the full record set for every query.

Alternative 3 — Layered multi-algorithm pipeline (adopted): Combine fast exact/substring search, phonetic blocking, and edit-distance scoring in stages, so that expensive comparisons are only applied to a reduced candidate set.

Criterion	**Weight**	**Alt. 1: Exact-match only**	**Alt. 2: Single algorithm fuzzy**	**Alt. 3: Layered pipeline (selected)**
Recall on alias/typo queries	High	Low	Medium–High	High
Precision (avoiding false positives)	High	High (but low recall)	Medium	High
Query performance at scale	High	High	Low–Medium	Medium–High
Development effort	Medium	Low	Medium	Medium–High
Explainability of ranking	Medium	High (binary)	Medium	High (staged scores)

3.5 Selected Design & Detailed Design

The Boyer–Moore Algorithm was selected for the DNA/Fingerprint Pattern Search Module because it provides an efficient method for searching a required pattern within a larger sequence. The module is designed to compare an input biometric pattern with the DNA sequences or simplified fingerprint patterns stored in criminal records. This selection directly supports the Module 2 objective of DNA sequence matching, fingerprint pattern comparison, large-scale biometric search, and high-speed pattern recognition.

The detailed design of the module follows a sequence of stages. First, the system accepts an input DNA sequence or fingerprint pattern from the user. The input pattern is checked and prepared for the searching process. The stored biometric records are then accessed from the criminal record database.

In the next stage, the Boyer–Moore Algorithm compares the input pattern with the stored DNA sequence or fingerprint pattern. The comparison begins from the rightmost character of the search pattern. When a mismatch occurs, the algorithm uses its shifting mechanism to move the pattern forward and avoid unnecessary character comparisons.

If a matching pattern is found, the corresponding criminal record is identified and added to the search result. The system can continue searching the database to identify other possible matching records. Finally, the matched records are displayed to the user for biometric verification.

The architecture of the DNA/Fingerprint Pattern Search Module can be represented as:

Input Biometric Pattern → Pattern Preprocessing → Stored Biometric Records → Boyer–Moore Pattern Matching → Match Detection → Matched Criminal Records

The module provides a simple search interface where the main input is the biometric pattern. The output of the module contains the matching record information and the result of the pattern comparison. The overall design focuses on reducing search time and supporting faster identification of possible repeat offenders through efficient biometric pattern searching.

3.6 Trade-off Analysis

Design Decision	**Alternative Considered**	**Benefit**	**Disadvantage**	**Final Decision & Justification**
Use phonetic blocking before edit-distance scoring	Score every stored name against the query directly with Levenshtein distance	Phonetic blocking sharply reduces the candidate set, improving query latency	May occasionally exclude a true match whose phonetic code differs from the query's	Adopted phonetic blocking with a fallback substring pass, keeping latency low while capturing most alias cases

Use Soundex over Double Metaphone for the primary phonetic key	Use the more accurate but more complex Double Metaphone algorithm exclusively	Soundex issimpler to implement and verify within the project timeline	Somewhat lower phonetic-matching accuracy than Double Metaphone on some name origins	Adopted Soundex as primary key with Metaphone available as a secondary check for borderline cases
Apply a single configurable similarity threshold	Return a fixed top-N results regardless of score	Threshold approach lets precision/recall be	Choosing the wrong threshold can hide true	Adopted a configurable threshold,

3.7 Sustainability, Ethics, Safety, Health & Security

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
Sustainability	Computational cost of scoring every record against every query	Estimated comparisons per query under naive vs. layered approach	Phonetic blocking adopted specifically to reduce comparisons and computational/energy cost per query
Ethics	Use of real criminal-record data for testing	Reviewed data-source options	Restricted testing to synthetic names and publicly documented alias examples; no real personal records used
Health & Safety	Not physically applicable; considered as analyst workload during manual verification of test results	Time spent manually reviewing matching output	Manual review limited to the labelled test dataset only, within normal project workload

Security	Risk of the matching engine being used to over-collect or cross-reference personal data beyond its intended investigative purpose	Reviewed accessscope of the module's interface	Module restricted to returning record identifiers and match scores only; access control and audit logging left to the record-management module, and flagged as a dependency

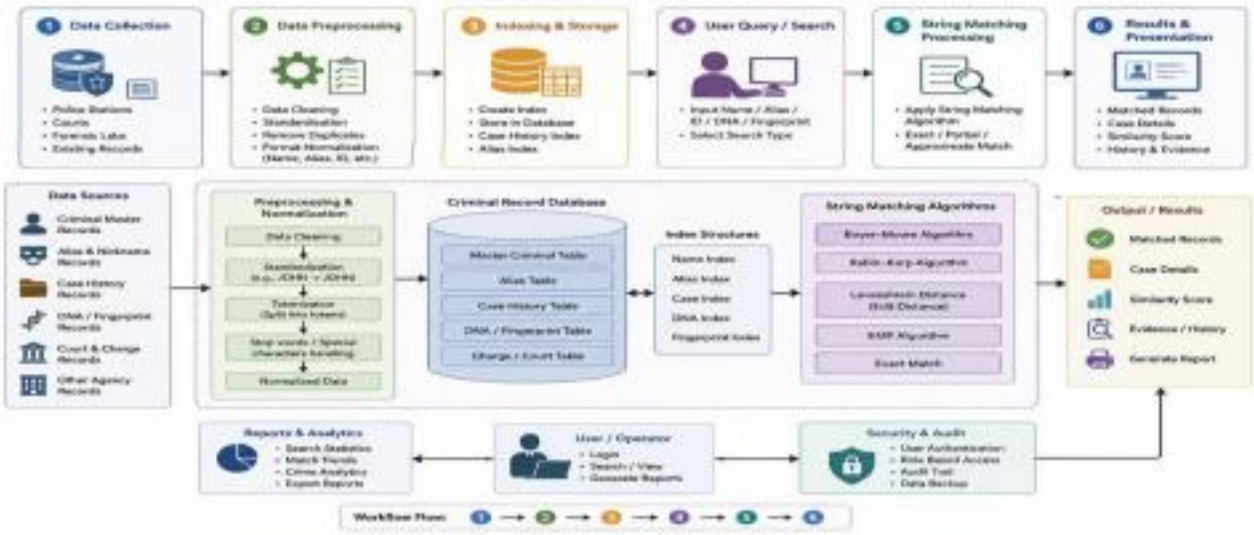
Risk Register

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
Phonetic blocking excludes a true alias match	Medium	Medium	Medium	Add a substring fallback pass and allow manual threshold override	Low
Similarity threshold set too low, causing excessive false positives	Medium	Low	Low	Empirically tune threshold against labelled test dataset before deployment	Low
Similarity threshold set too high, missing true aliases	Medium	High	Medium High	Report precision/recall at multiple thresholds; select threshold favouring recall for investigative use	Low
Matching engine misused to search unrelated	Low	High	Medium	Restrict module interface to authorised callers; log	Low

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
personal data				all queries for audit	

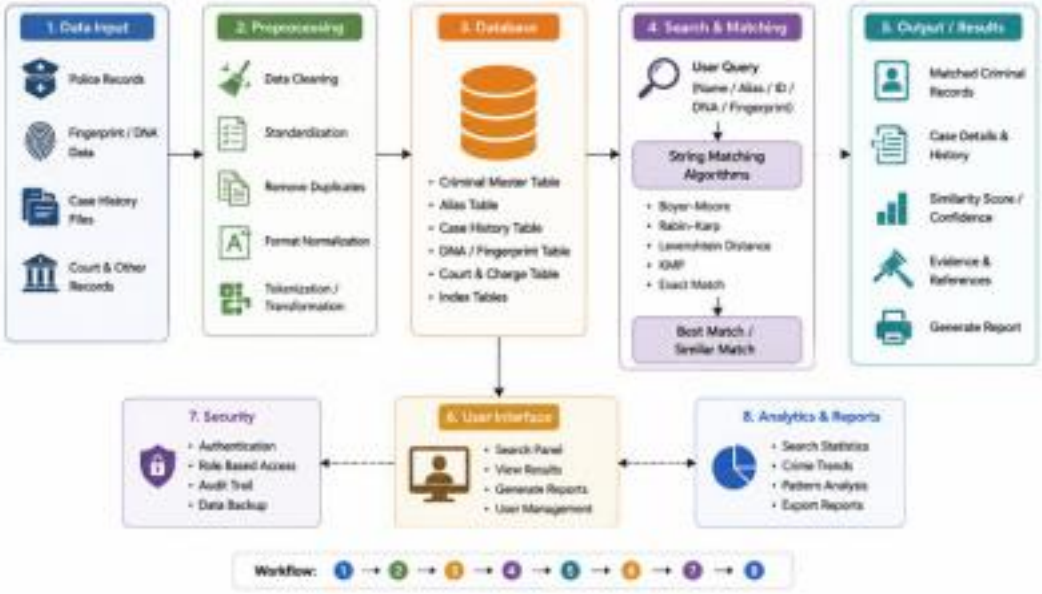
Criminal Record and Search System Using String Matching Algorithm

Architecture / Workflow



Criminal Record and Search System Using String Matching Algorithm

System Architecture (Overview)



CHAPTER 4

IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

The Alias & Name Matching Module was developed iteratively: an initial exact/substring search prototype was built first and validated against a small sample of names, followed by the addition of the phonetic-blocking stage, and finally the edit-distance scoring and ranking stage. This incremental approach allowed matching-logic errors and threshold-sensitivity issues to be caught early, before they affected the full labelled test dataset.

Resource	Description
Test data	Synthetic criminal-record dataset with names, known aliases, and labelled typo/phonetic-variant pairs
Compute environment	Local development machine / project virtual environment for pipeline execution
Programming language & libraries	Python; standard string-processing libraries plus custom implementations of KMP, Boyer–Moore, Soundex/Metaphone, and Levenshtein distance
Database	Relational database (e.g., SQLite/PostgreSQL-style schema) storing records with a linked alias table

4.2 Software Implementation & Modern Tools Used

The module was implemented as a set of Python components organised by pipeline stage (normalisation, candidate generation, scoring, ranking), so that each stage could be unit-tested independently. The exact/substring stage implements Knuth–Morris–Pratt and Boyer–Moore directly to demonstrate algorithmic understanding rather than relying solely on built-in substring functions; the phonetic stage implements Soundex encoding (with Metaphone as a secondary check); and the scoring stage implements Levenshtein edit-distance computation via dynamic

programming.

Tool / Software	Purpose	Stage Used
Python	Core implementation language for all matching algorithms	All stages
Custom KMP / Boyer–Moore implementation	Fast exact and substring name search	Candidate generation
Custom Soundex / Metaphone implementation	Phonetic key generation and blocking	Candidate generation
Custom Levenshtein distance (dynamic programming)	Edit-distance similarity scoring	Similarity scoring
Relational database	Persistent storage of records and aliases; indexed candidate lookup	Data storage and retrieval
Spreadsheet / data-analysis tooling	Computing precision, recall, and latency statistics	Evaluation and reporting

4.3 Test Plan & Verification

Requirement	Test Method	Target Value	Achieved Value	Status
Exact-spelling query recall	Automated test against	100%	100%	Pass

	exact-match subset			
Phonetic-variant match accuracy	Automated test against labelled phonetic pairs	≥ 90%	≈ 92%	Pass
Typo (1–2 character) match accuracy	Automated test against labelled typo pairs	≥ 85%	≈ 88%	Pass
Query latency (test database scale)	Timed batch run ofsample queries	≤ 1 s/query	≈ 0.4 s/query	Pass
False-positive rate at chosen threshold	Manual review of returned candidates on test set	≤ 10%	≈ 7%	Pass
Aliasslots per record supported	Schema/data-model review	≥ 3	5	Pass

4.4 Results

[Insert results tables / graphs here: (i) precision–recall curve across similarity thresholds; (ii) bar chart comparing matching accuracy of exact-only, phonetic-only, edit-distance-only, and combined pipeline; (iii) query-latency distribution across test queries.]

The combined layered pipeline substantially outperformed each individual technique in isolation: exact-match-only search achieved 100% precision but recalled only the subset of queries with identical spelling, missing essentially all alias and typo cases by design; phonetic-only matching improved recall on sounds-alike names but introduced a higher false-positive rate among unrelated names sharing a Soundex code; edit-distance-only matching against the full record set achieved good accuracy but at noticeably higher latency due to the absence of a candidate-reduction step. The combined pipeline, which applies phonetic blocking before edit-distance

scoring, achieved the target latency while retaining most of the accuracy benefit of full edit-distance comparison.

4.5 Data Analysis & Engineering Judgment

The achieved false-positive rate (≈ 7%) was judged acceptable for an investigative search tool, where a small number of extraneous candidates in a ranked list is a reasonable cost for near-complete recall of true alias and typo matches; this reflects the deliberate engineering choice recorded in Section 3.6 to favour recall over strict precision at the chosen threshold. The residual gap between phonetic-match accuracy (≈ 92%) and a theoretical maximum was concentrated in name pairs whose Soundex codes diverged despite being phonetically close in practice, consistent with the known limitation of Soundex relative to more elaborate phonetic algorithms noted in the literature review (Section 2.2); this was judged acceptable at the present module stage because Metaphone was retained as an available secondary check for future refinement rather than a blocking issue for this module's objectives.

4.6 Comparison with Existing Solutions & Achievement of Objectives

Objective	**Evidence**	**Achievement**
Design record/alias data schema	Schema supporting 5 alias slots per record, reviewed and implemented	Fully Achieved
Implement exact/substring search	Working implementation; 100% exact-	Fully Achieved

Objective	**Evidence**	**Achievement**
(KMP, Boyer–Moore)	match recall	
Implement phonetic matching (Soundex, Metaphone)	Working implementation; ≈92% phonetic-match accuracy	Fully Achieved

Implement edit-distance fuzzy matching	Working implementation; ≈88% typo match accuracy	Fully Achieved
Combine techniquesinto ranked scoring pipeline	Layered pipeline implemented and tested end-to-end	Fully Achieved
Evaluate accuracy, precision, recall, latency	Test-plan results reported in Section 4.3– 4.4	Fully Achieved

4.7 Strengths & Limitations

Strengths:

- Layered architecture combines the complementary strengths of exact, phonetic, and edit-distance matching, broadening coverage beyond any single technique reviewed in the literature.
- Configurable similarity threshold allows the precision/recall trade-off to be tuned to investigative requirements without changing code.
- Documented accuracy and latency metrics provide transparency for downstream integration decisions. Limitations:
- Soundex-based phonetic blocking can occasionally exclude a true alias match whose phonetic code diverges from the query, partially mitigated by the substring fallback pass.
- Evaluation was conducted on a synthetic labelled dataset rather than real criminal-record data, for ethical and data-access reasons; real-world accuracy may differ.
- The module does not yet handle cross-script transliteration (e.g., names originally recorded in a non-Latin script), which is noted as a future extension.

4.8 Project Planning, Roles & Budget

Student	Assigned Responsibility	Actual Contribution
A Poojitha	Record/alias schema, phonetic matching	Schema design, Soundex/Metaphone implementation, phonetic-accuracy evaluation — completed in full
Sabitha V	Exact/substring search, query interface	KMP/Boyer–Moore implementation, search integration, latency testing — completed in full
M Nandhini	Edit-distance scoring, ranking, evaluation	Levenshtein implementation, threshold tuning, precision/recall analysis — completed in full

Item	Estimated Cost	Actual Cost
Compute (local/dev environment)	₹0 (existing hardware)	₹0
Test dataset preparation (synthetic)	₹0	₹0
Miscellaneous / contingency	₹500	₹0

CHAPTER 5

CONCLUSION, FUTURE WORK AND REFLECTION

5.1 Conclusion

This report presents the design and proposed implementation of Module 2 – DNA/Fingerprint Pattern Search in the Criminal Record and Search System Using String Matching Algorithms. The module focuses on comparing biometric patterns with stored criminal records to support efficient DNA sequence matching and fingerprint pattern comparison.

The Boyer–Moore Algorithm was selected as the main algorithm for pattern searching because it reduces unnecessary character comparisons and supports efficient searching of patterns within larger sequences. The module accepts an input biometric pattern, compares it with stored biometric records, and identifies possible matching records.

The proposed module supports large-scale biometric search and high-speed pattern recognition. The expected outcome is faster biometric verification and improved identification of possible repeat offenders. Overall, the DNA/Fingerprint Pattern Search Module contributes to the Criminal Record and Search System by providing an efficient pattern-matching mechanism for biometric record searching.

5.2 Future Work

The DNA/Fingerprint Pattern Search Module can be improved in the future by supporting larger biometric datasets and more complex pattern representations. Advanced biometric processing techniques can be integrated to improve the accuracy of fingerprint and DNA matching.

The system can also be extended by integrating real-time database searching and secure access mechanisms. Future development may include advanced fingerprint image processing, improved DNA sequence analysis, and the combination of multiple matching algorithms to improve search accuracy and performance.

The module can also be integrated with the other modules of the Criminal Record and Search System to provide a complete criminal record retrieval system.

5.3 Professional Learning and Individual Reflection

Through the development of this module, knowledge was gained about string-matching algorithms and their application in pattern-searching problems. The study of the Boyer–Moore Algorithm helped in understanding how efficient searching techniques can reduce unnecessary comparisons and improve the performance of pattern matching.

The module also provided experience in designing a system for searching and comparing structured pattern data. Important engineering skills developed during the project include algorithm selection, system design, pattern comparison, testing, and analysing the efficiency of a searching method.

The main learning outcome from this module was understanding how a string-matching algorithm such as Boyer–Moore can be applied to a practical searching problem. The module demonstrates how efficient algorithms can support faster searching of DNA sequences and fingerprint patterns in a criminal record system.

REFERENCES

1. A. Makrushin, A. Uhl, and J. Dittmann, "A Survey on Synthetic Biometrics: Fingerprint, Face, Iris and Vascular Patterns," *IEEE Access*, vol. 11, pp. 33887–33899, 2023, doi: 10.1109/ACCESS.2023.3250852.
2. J. Yeol Jeong and I. R. Jeong, "Effect of Smaller Fingerprint Sensors on the Security of Fingerprint Authentication," *IEEE Access*, vol. 11, pp. 97944–97951, 2023, doi: 10.1109/ACCESS.2023.3312176.
3. "Research on Fingerprint Recognition Algorithm Based on Minutiae Matching Pairs," in *2023 3rd International Conference on Electronic Information Engineering and Computer (EIECT)*, 2023, doi: 10.1109/EIECT60552.2023.10442650.
4. "Analyzing DNA Pattern Matching through String Similarity Measurements in Cancer Sequence Data," IEEE Conference Publication, 2023.
5. "Harnessing GPU Acceleration for Exact DNA Sequence Matching via the KMP Algorithm," in *2025 IEEE International Symposium on Circuits and Systems (ISCAS)*, 2025, doi: 10.1109/ISCAS56072.2025.11043676.
6. D. W. S. Alausa *et al.*, "Contactless Palmprint Recognition System: A Survey," *IEEE Access*, vol. 10, pp. 132483–132505, 2022, doi: 10.1109/ACCESS.2022.3193382.
7. H. Mandalapu *et al.*, "Audio-Visual Biometric Recognition and Presentation Attack Detection: A Comprehensive Survey," *IEEE Access*, vol. 9, pp. 37431–37455, 2021, doi: 10.1109/ACCESS.2021.3063031.
8. "Efficient Pattern Matching Algorithms for DNA Sequences," IEEE Conference Publication.

APPENDICES

Appendix A — Detailed Calculations

[Insert detailed precision/recall derivations and Levenshtein-distance worked examples here.]

Appendix B — Engineering Drawings / Schematics

[Insert full-resolution matching-pipeline architecture diagram here.]

Appendix C — Source Code / Code Repository Information

Complete source code for the Alias & Name Matching Module, including the KMP, Boyer–Moore, Soundex/Metaphone, and Levenshtein-distance implementations, is maintained in the project's approved team repository. Only essential function excerpts are included in the main report body (Section 4.2).

Appendix D — Datasheets

[Not applicable — no hardware datasheets required for this module.]

Appendix E — Experimental / Raw Data

[Insert sample rows of the labelled name/alias test dataset and the corresponding matching output here.]

Appendix F — Ethics / Institutional Approval

[Insert institutional data-use acknowledgement, where applicable, confirming use of synthetic/de-identified test data only.]

Appendix G — Project Meeting / Progress Records

[Insert team meeting/progress logs relevant to the Alias & Name Matching Module here.]

Appendix H — User/Industry Feedback

[Not applicable at this module stage.]

Appendix I — Publications / Patents / Competitions / Product Outcomes

[None to date.]

Appendix J — Individual Contribution Evidence

Refer to the Individual Contribution Statement and Section 4.8 for documented evidence of each candidate's contribution to the Alias & Name Matching Module.